

Mount namespace privado em um runtime estilo Docker

Construção da feature de isolamento de propagação de mounts em um projeto from scratch com control plane em Python, cola em Cython e data plane em C++.

ESCOPO

Definições, contexto no Docker, arquitetura alvo, LLD, DDD, GoF, requisitos e roadmap

CONTEXTO DO PROJETO

Runtime estilo Docker construído do zero

FEATURE ESTUDADA

Mount namespace privado para impedir propagação de mounts para fora

DATA

12 de março de 2026

Resumo executivo

O ponto central desta feature é que um `CLONE_NEWNS` isolado não basta. O runtime precisa tratar propagation de forma explícita, porque a nova namespace herda a semântica dos mounts existentes. Em hosts modernos, isso pode manter a árvore como `shared` ou `slave`; logo, um bind mount interno ainda pode vazar se o backend não remountar a árvore ou os mounts relevantes como `private` ou `slave` e não preparar corretamente o rootfs [1,2,3,11,12].

Tese central

Minha recomendação é tratar `rprivate` como contrato externo padrão do produto e reservar `rslave` como estratégia interna de bootstrap ou perfil explícito de compatibilidade. O backend deve materializar essa decisão com `parent-private` no rootfs, `self-bind`, verificação por `mountinfo` e, idealmente, espaço para evolução futura com `open_tree`, `move_mount` e `mount_setattr` [4,8,9,13,14].

Sumário

1. Introdução e tese de implementação
 2. Fundamentos: mount namespace, bind mounts e mount propagation
 3. Como Docker, OCI e runc tratam o problema
 4. Tradução para o projeto em Python, Cython e C++
 5. System Design proposto
 6. Low Level Design
 7. Domain-Driven Design
 8. Padrões de Design (GoF)
 9. Requisitos funcionais e não funcionais
 10. Plano de implementação por fases
 11. Estratégia de testes, validação e observabilidade
 12. Conclusão
- Referências

1. Introdução e tese de implementação

Esta pesquisa trata de uma feature que parece simples na superfície - “criar um mount namespace privado” - mas que, na prática, expõe quase toda a complexidade semântica de um runtime de containers. O projeto descrito pelo usuário tem uma separação arquitetural muito clara: control plane, metadados, API e reconciliação em Python; integração binária e marshaling em Cython; data plane, syscalls, abstrações próximas ao kernel e execução do bootstrap do container em C++. Essa divisão é tecnicamente saudável, desde que a fronteira entre as camadas seja rígida o suficiente para impedir que detalhes de baixo nível contaminem o modelo de domínio e, ao mesmo tempo, que decisões de domínio não fiquem escondidas dentro do código nativo.

O problema específico de “mount namespace privado” exige rigor terminológico desde a primeira linha. Um novo mount namespace não equivale, automaticamente, a um namespace privado. A operação de criar um namespace novo clona a visão da árvore de mounts do processo chamador, e os mounts herdados mantêm sua semântica de propagation até que ela seja alterada explicitamente. Em outras palavras, o isolamento de nomes e pontos de montagem não apaga o histórico de propagation da árvore copiada. Esse detalhe é o ponto de partida correto para qualquer design sério, porque é exatamente nele que nascem vazamentos de bind mounts entre container e host quando o runtime apenas chama `unshare(CLONE_NEWNS)` ou `clone3(..., CLONE_NEWNS)` e assume que o trabalho terminou [1,2,3].

Em distribuições atuais, esse risco é ainda mais concreto. A documentação de `mount_namespaces(7)` observa que sistemas modernos inicializados por `systemd` costumam remountar os mounts existentes como `MS_SHARED` durante o boot. Isso significa que a nova namespace pode começar com uma cópia de mounts compartilhados, não de mounts privados. A própria mesma documentação nota que ferramentas como `unshare(1)` passaram a tornar os mounts privados no namespace recém-criado, mas isso é uma conveniência da ferramenta de espaço de usuário; quem usa syscalls diretamente precisa fazer o remount de propagation por conta própria. Portanto, um runtime caseiro não deve confiar em comportamentos implícitos de wrappers do sistema operacional, e sim tornar a policy de propagation um passo explícito e auditável do bootstrap [1,15].

Princípio operacional que guia o restante do relatório

O design deve separar três coisas que muita implementação mistura: a intenção do produto, a semântica do contrato exposto ao usuário e a mecânica interna do bootstrap. A intenção do produto, neste caso, é impedir propagação de mounts para fora. O contrato exposto deve, na minha avaliação, ser `rprivate` por padrão. A mecânica interna pode usar um estágio intermediário `rslave` por compatibilidade, desde que esse detalhe nunca escape para a API como comportamento implícito e que o estado final observado seja coerente com o contrato assumido.

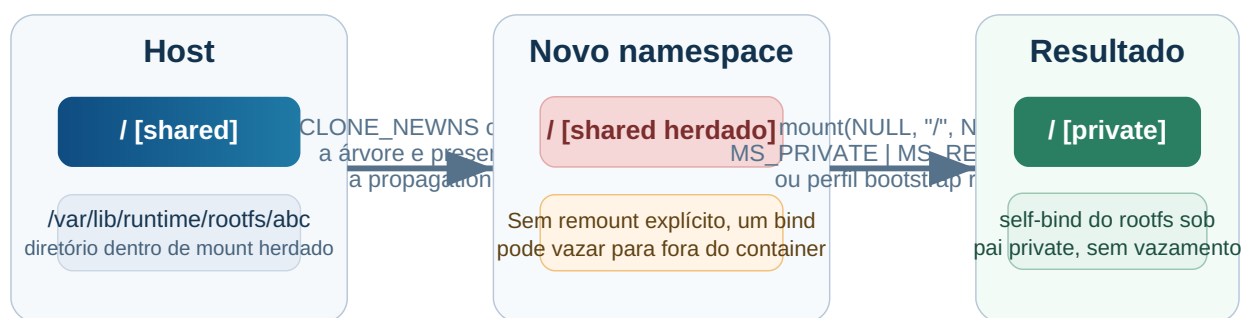
O relatório segue uma metodologia híbrida. Primeiro, consolida definições e semânticas a partir da documentação canônica do kernel Linux, dos man pages, da documentação oficial do Docker, da especificação OCI e do código do runc. Em seguida, traduz essas informações para decisões de engenharia apropriadas ao seu projeto, cobrindo System Design, Low Level Design, Domain-Driven Design, padrões GoF, requisitos funcionais e não funcionais, e um roadmap de implementação. Onde a documentação oficial descreve comportamento e não prescrição arquitetural, o texto assume posição e propõe uma solução. Essas propostas são deliberadas: não são apenas resumo de documentação, mas um plano tecnicamente opinado para um runtime robusto.

2. Fundamentos: mount namespace, bind mounts e mount propagation

2.1 O que um mount namespace realmente isola

O mount namespace isola a visão da árvore de mounts, não o armazenamento em si. Quando um processo entra em um novo mount namespace, ele passa a observar uma cópia da lista de mounts do namespace de origem. A partir daí, operações de mount e unmount podem divergir entre os namespaces, mas os mounts copiados carregam consigo a semântica de propagation que possuíam no momento da clonagem. A documentação do kernel é clara ao afirmar que os tipos de propagation são preservados na criação do novo namespace, a menos que sejam alterados depois. O resultado prático é simples: o namespace recém-criado pode estar isolado do ponto de vista de identidade de mount points e, ao mesmo tempo, não estar privado do ponto de vista de propagation [1,2].

Esse é o motivo pelo qual a pergunta correta nunca deve ser “criei um `CLONE_NEWNS`?” e sim “qual é a propagation efetiva dos mounts que ficaram no namespace depois da clonagem?”. Em ambientes com raiz compartilhada, o mount root do novo namespace pode continuar `shared` ou ser automaticamente reduzido para `slave` em certos cenários menos privilegiados. A distinção importa porque a feature estudada não se resume a “ter um namespace diferente”; ela exige que eventos de mount feitos dentro do container não atravessem a fronteira e reapareçam fora dele.



O ponto central é que um mount namespace novo não é, por si só, um mount namespace privado.

Figura 1. Um novo mount namespace herda a árvore e sua propagation; o isolamento desejado só aparece depois do remount explícito da propagation e do preparo correto do rootfs.

2.2 O que é mount propagation

Mount propagation regula o fluxo de eventos de mount e unmount entre mounts relacionados. Ela não controla leitura e escrita de arquivos; controla o aparecimento e desaparecimento de outros mounts no interior da árvore. O kernel documenta quatro estados principais: `shared`, `slave`, `private` e

unbindable. Um mount **shared** participa de um peer group e replica eventos para seus pares; um mount **slave** recebe eventos do mount mestre, mas não replica de volta; um mount **private** nem recebe nem propaga eventos; um mount **unbindable** é privado e, adicionalmente, não pode ser usado como origem de bind mount [2,3].

As formas recursivas, como **rprivate**, **rslave** e **rshared**, não são novos estados lógicos; são a combinação da mudança de propagation com **MS_REC**, que aplica a transição a uma árvore inteira de mounts. Esse ponto precisa estar refletido no modelo de domínio e no contrato de API. Em uma implementação séria, **MS_PRIVATE** não é um detalhe de opção textual; é uma decisão semântica aplicada sobre um nó da árvore e, frequentemente, recursivamente sobre a subárvore inteira [3,4].

MODO	RECEBE EVENTOS DO PAI	PROPAGA PARA FORA	LEITURA DE PROJETO
private / rprivate	Não	Não	É a semântica mais previsível para isolamento forte. O runtime não recebe nem envia eventos de mount ou unmount. Use como default do produto.
slave / rslave	Sim	Não	É útil como estágio de bootstrap ou quando o container precisa observar submounts do host sem poder vazar mounts de volta. É mais compatível com algumas topologias herdadas.
shared / rshared	Sim	Sim	É poderoso, mas perigoso. Deve ser opt-in, restrito a bind mounts específicos e bloqueado por policy para a maioria dos workloads.
unbindable / runbindable	Não	Não	Além de privado, não pode ser origem de bind mount. É útil para reduzir superfícies específicas, mas não deve ser tratado como substituto geral de private .

Um detalhe importante para seu projeto é que “privado” não precisa significar “cego ao host” em todos os perfis. Se o requisito de produto é apenas impedir propagação para fora, um perfil **slave** já basta do ponto de vista estritamente direcional. Mas ele mantém um acoplamento com a topologia externa que aumenta a complexidade cognitiva e a dificuldade de depuração. Por isso, embora eu trate **rslave** como opção valiosa de bootstrap ou compatibilidade, considero **rprivate** o contrato mais limpo para o default do runtime.

2.3 Bind mounts e por que eles são o foco do risco

Bind mount é a operação que torna um diretório ou arquivo já existente visível em outro ponto da árvore. Em **mount (2)**, um bind mount simples replica um ponto de vista para a mesma subárvore; quando combinado com **MS_REC**, ele inclui submounts existentes, exceto os que forem **unbindable**. Para um

runtime de containers, bind mounts aparecem em pelo menos três lugares: no self-bind do próprio rootfs, na montagem de pseudo-filesystems como `/proc` e `/dev`, e nos binds declarados pelo usuário para injetar diretórios do host dentro do container [3,8].

O problema é que a criação de um novo bind mount não ocorre em abstrato; ela acontece dentro de uma árvore com regras de propagation já ativas. A documentação de `mount_namespaces(7)` descreve que novos mounts podem entrar em peer groups compartilhados quando são replicados ou quando novos bind mounts são criados a partir de mounts compartilhados. Em termos práticos, se o rootfs estiver sobre um pai ainda compartilhado, um self-bind aparentemente inofensivo pode propagar para fora. É exatamente por isso que o `runc` força o mount pai do rootfs a ficar privado antes de fazer o bind do rootfs sobre si mesmo [1,11,12].

Erro clássico de implementação

Uma implementação ingênua costuma fazer: criar namespace, bindar o rootfs, montar `proc`, montar diretórios do usuário e seguir adiante. Isso funciona na máquina do desenvolvedor até o dia em que o host tem uma raiz compartilhada. A partir daí, surgem sintomas aparentemente aleatórios: bind mounts “fantasmas” no host, falhas de `pivot_root` e comportamentos diferentes entre distribuições. O problema não é aleatório; é uma consequência direta de propagation não tratada explicitamente.

2.4 Como verificar o estado real: `/proc/<pid>/mountinfo`

Em um runtime desse tipo, o estado real do mount graph mora no kernel, não no banco de metadados. A fonte de verdade para diagnóstico e reconciliação é `/proc/<pid>/mountinfo`. A documentação de `proc.txt` descreve o formato e, em especial, os campos opcionais que aparecem antes do separador `-`. Neles, mounts compartilhados exibem marcações como `shared:X`; mounts slave exibem `master:X` e eventualmente `propagate_from:X`; mounts unbindable exibem `unbindable`. Um mount privado, em geral, não apresenta essas tags. Isso não dispensa teste funcional, mas já permite uma verificação objetiva de conformidade entre o plano desejado e o estado observado [5].

Esse detalhe muda a arquitetura de observabilidade do sistema. Você não quer uma API que apenas devolva a configuração pedida pelo usuário. Você quer uma API de inspeção que exponha o que o kernel realmente materializou. Em um sistema com aspiração de produção, `mountinfo` precisa ser capturado como snapshot ao fim do bootstrap, normalizado em uma estrutura estável e persistido como “observed state”, ao lado do “desired state” produzido pela API.

3. Como Docker, OCI e runc tratam o problema

Vale separar as camadas do ecossistema original. O Docker Engine expõe a arquitetura cliente-servidor conhecida: CLI e APIs conversam com o daemon, e o daemon delega a execução de containers para componentes mais próximos do runtime. A documentação oficial descreve que o Docker usa o `containerd` e que, por padrão, o `containerd` usa o `runc`. Esse detalhe é importante porque a semântica real de namespaces e mounts não é decidida no nível do CLI; ela é materializada no runtime baixo nível que fala com o kernel [6,7].

3.1 O contrato exposto pelo Docker

Na superfície de produto, o Docker adota uma postura conservadora para propagation. A documentação de bind mounts afirma que a propagation padrão de bind mounts é `rprivate` e que a customização de propagation só existe para bind mounts em Linux. A documentação de volumes vai além: volumes usam bind propagation `rprivate` e esse comportamento não é configurável. Essa escolha transmite um princípio de design útil para o seu projeto: a maior parte dos usuários quer isolamento previsível; quem precisa de propagation compartilhada deve pedir isso explicitamente e entender o impacto dessa escolha [8,9].

Há uma consequência importante aqui. Quando se diz que Docker “usa `rprivate`”, isso não significa que toda a implementação interna seja um único remount global e pronto. O produto expõe defaults de propagation para mounts de usuário e volumes, enquanto o runtime interno pode usar estágios intermediários para preparar o rootfs com segurança. Essa distinção entre contrato externo e mecânica interna é exatamente a distinção que seu projeto precisa preservar desde o início.

3.2 O papel da OCI e do rootfsPropagation

A especificação OCI para Linux prevê o campo `rootfsPropagation`, permitindo que o rootfs seja tratado como `shared`, `slave`, `private` ou `unbindable`. O detalhe fino é que isso controla a propagation do mount root do container, não apenas dos bind mounts individuais. A especificação também deixa claro que, mesmo quando se usa `shared`, isso significa pertencer a um novo peer group; não implica, por definição, “compartilhar com o host” de maneira irrestrita. Ainda assim, do ponto de vista de produto, permitir rootfs compartilhado aumenta a carga de compatibilidade, então essa capacidade deve existir como recurso avançado, não como default [10,11].

3.3 O que o runc realmente faz

O código do `runc` é especialmente instrutivo porque ele mostra a diferença entre semântica final e bootstrap seguro. Em `specconv/spec_linux.go`, as strings de propagation da OCI são mapeadas para flags de kernel: `rprivate` vira `MS_PRIVATE | MS_REC`, `rslave` vira `MS_SLAVE | MS_REC`, e assim por diante. Já em `rootfs_linux.go`, a preparação do rootfs, quando não há valor explícito no config, começa com `MS_SLAVE | MS_REC` sobre `/`. Em seguida, o runtime torna privado o mount pai do rootfs, faz o self-

bind do rootfs e só depois executa `pivot_root` ou `chroot`. O comentário do código é inequívoco: o pai precisa ser privado porque `pivot_root` falha em pai compartilhado e porque o bind mount do rootfs poderia vaziar para o namespace pai caso o mount continuasse compartilhado [11,12].

Essa sequência responde a uma dúvida frequente de quem estuda o tema. Se o objetivo final é isolamento forte, por que o `runc` começa com `rslave` e não com `rprivate`? A resposta é compatibilidade operacional. `rslave` impede propagação para fora, que é o requisito mínimo de contenção, mas ainda permite que certas alterações do host entrem no namespace durante a fase inicial. Em contrapartida, o `runc` precisa fazer o passo adicional do pai do rootfs privado para garantir que o self-bind não vaze. Para o seu runtime, isso sugere duas estratégias válidas: um perfil “Docker-compatível”, que replica essa disciplina, e um perfil “isolamento estrito”, que assume `rprivate` como default desde o início.

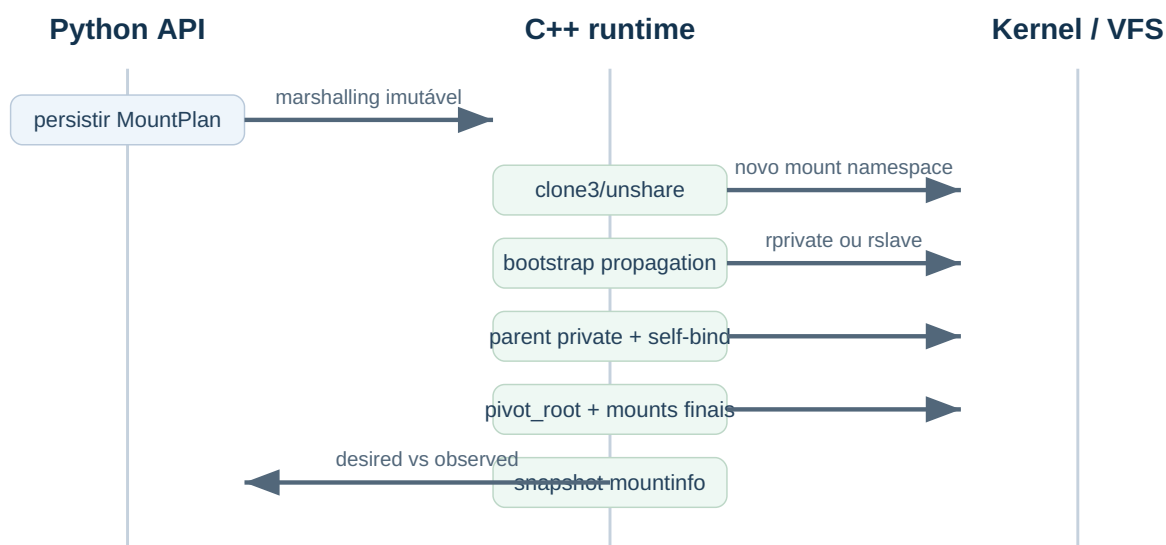


Figura 2. Sequência de bootstrap recomendada: desired state em Python, bootstrap em C++ e verificação do estado observado antes do container iniciar de fato.

3.4 Restrições e edge cases que vêm do runtime original

O código do `runc` também deixa uma pista de segurança importante: ele rejeita `rootfsPropagation` em modo `private` ou `rprivate` quando a configuração pede `NoPivotRoot`. Essa recusa não é perfumaria; ela indica que determinadas combinações, embora sintaticamente possíveis, não produzem isolamento seguro sem a troca real da raiz. Para o seu projeto, a consequência direta é que a API precisa validar combinações de opção à luz das restrições do mecanismo subjacente, e não apenas repassar strings para o backend [11].

Outro edge case relevante aparece na própria documentação de `mount_namespaces(7)` e de `mount_setattr(2)`: em namespaces menos privilegiados, especialmente quando um novo user namespace está envolvido, mounts compartilhados herdados podem ser reduzidos a `slave` e certas alterações de propagation podem ficar bloqueadas. Portanto, `rootless` não deve ser vendido como simples “o mesmo comportamento sem root”; ele merece um perfil de capacidade separado, com garantias explícitas, eventuais degradações e mensagens de erro orientadas a causa [1,4].

4. Tradução para o projeto em Python, Cython e C++

A arquitetura proposta pelo usuário é coerente com a natureza do problema. Mount namespaces, bind mounts, propagation flags e troca de raiz pertencem ao domínio do data plane e exigem código que opere confortavelmente no nível de syscalls, file descriptors, `errno` e rollback transacional. Já a interpretação do que o usuário pediu, a geração do plano, a persistência dos metadados, a observabilidade e a reconciliação entre desejo e estado materializado pertencem claramente ao control plane. O erro mais comum em projetos assim é confundir a fronteira: deixar regras de produto escaparem para o C++ porque “é lá que tudo acontece”, ou empurrar detalhes de syscalls para o Python porque “é mais fácil mudar”. As duas escolhas produzem sistemas difíceis de evoluir.

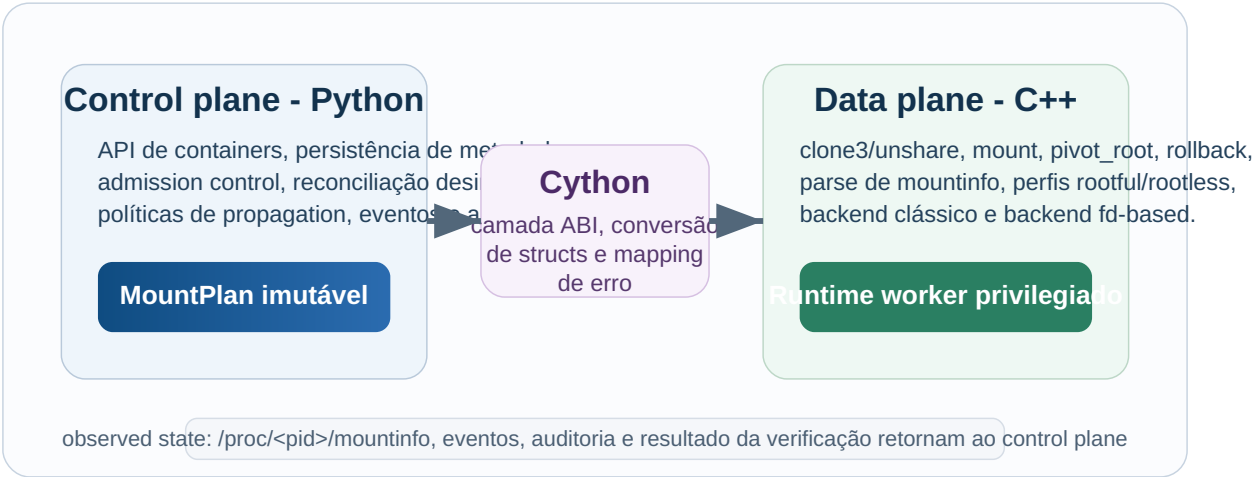


Figura 3. Arquitetura-alvo: o control plane decide, o data plane materializa, e o estado observado retorna ao Python como fato de runtime.

CAMADA	RESPONSABILIDADES LEGÍTIMAS	RESPONSABILIDADES QUE EU EVITARIA
Python	API, autenticação, persistência, modelo de domínio, compilação do MountPlan, admission control, reconciliação desired vs observed, inspeção e auditoria.	Chamar <code>mount</code> , <code>pivot_root</code> ou mutar namespace dentro do processo multithreaded que serve a API.
Cython	ABI estável, conversão de tipos imutáveis, liberação do GIL, mapeamento de <code>errno</code> e códigos nativos para exceções Python.	Embutir regras de domínio, policies de propagation, lógica de rollback ou branches de segurança difíceis de testar.

CAMADA	RESPONSABILIDADES LEGÍTIMAS	RESPONSABILIDADES QUE EU EVITARIA
C++	Execução do bootstrap, syscalls, rollback, parse de mountinfo, gerenciamento de FDs, backends de mount e compatibilidade com perfis rootful/rootless.	Persistência de negócio, regras de API, semântica de autorização ou decisões de produto que deveriam ficar visíveis no control plane.

4.1 O que deve viver no control plane

Eu colocaria em Python tudo o que representa linguagem ubíqua de produto. Isso inclui objetos como `ContainerSpec`, `MountSpec`, `MountPlan`, `PropagationPolicy`, `CapabilityProfile` e `ObservedMountTopology`. Também colocaria em Python o admission control que rejeita pedidos incompatíveis com a policy do produto, por exemplo: binds compartilhados em perfis que só permitem isolamento estrito, volumes configurados com propagation diferente de `rprivate`, ou combinações inseguras como `NoPivotRoot` com rootfs privado. Esse é o lugar correto para tornar as regras auditáveis, versionáveis e fáceis de testar.

No modelo operacional, o Python deve persistir um plano imutável antes do bootstrap. Isso significa que a API não entrega um conjunto de parâmetros soltos ao worker; ela entrega um artefato lógico fechado, com checksum, versão de schema e escolhas resolvidas. Em outras palavras, o C++ não deveria “descobrir” policy; ele deve apenas receber uma decisão já tomada e executá-la da forma mais segura possível.

4.2 O papel correto do Cython

O Cython deve ser tratado como cola de ABI, não como uma terceira camada de negócio. O objetivo dele é converter estruturas Python imutáveis para structs nativas estáveis, liberar o GIL em chamadas longas, repassar buffers sem cópias desnecessárias e transformar erros nativos em exceções Python ricas em contexto. Quanto mais regra de negócio entrar no Cython, pior fica a depuração e mais nebulosa se torna a propriedade do código. Meu conselho aqui é quase disciplinar: Cython deve ser chato, fino e previsível.

Na prática, eu adotaria um padrão em que o Python constrói um `LaunchPlan` serializável e, ao chamar o backend, o Cython apenas o converte para um `LaunchPlanNative`. A chamada ao runtime ocorreria com `with nogil`, e qualquer retorno do tipo `Result<T, LaunchError>` seria traduzido para uma exceção Python com campos estruturados: `syscall` falha, `errno`, path envolvido, propagation desejada, propagation observada e estágio em que a falha ocorreu.

4.3 O que deve viver no C++

O data plane em C++ deve concentrar todo o ciclo de vida do bootstrap: abertura de paths de forma segura, criação do processo filho, `clone3` ou `unshare`, remount da propagation, self-bind do rootfs, criação de mount points internos, montagem dos pseudo-filesystems, aplicação dos bind mounts declarados,

`pivot_root`, parse de `mountinfo`, rollback e sincronização de status com o pai. Além disso, é em C++ que eu colocaria a compatibilidade com dois backends: um backend clássico baseado em `mount(2)` e um backend mais moderno baseado na nova mount API do kernel [3,4,13,14].

Existe ainda uma decisão de processo que considero fundamental. O processo Python que atende API não deve chamar `fork` e continuar executando lógica Python no filho antes de `exec`. A janela entre `fork` e `exec` é notoriamente delicada em runtimes multithreaded e ainda mais quando o filho vai entrar em namespaces e operar mounts. O caminho mais limpo é usar um worker nativo dedicado, preferencialmente processo separado, e deixar o código C++ possuir o caminho do filho desde a criação até o ponto em que o estado já está materializado e o sync pipe foi respondido.

5. System Design proposto

Do ponto de vista de System Design, a feature deve ser modelada como um pipeline que transforma intenção declarativa em topologia de mounts verificada. O fluxo começa na API, passa por validação e compilação de um plano imutável, chega a um executor nativo que materializa a árvore de mounts no kernel e retorna um snapshot observado antes de o container ser marcado como pronto. Essa ordem é essencial porque evita um erro conceitual comum: considerar o bootstrap bem-sucedido apenas porque nenhuma syscall falhou. Em namespace e mounts, ausência de erro não basta; é necessário provar que a topologia final corresponde ao contrato pedido.

5.1 Componentes propostos

Eu dividiria o sistema em cinco componentes principais. O primeiro é a API de containers, em Python, que recebe a intenção do usuário, aplica autenticação e autorização e produz um `ContainerSpec`. O segundo é o compilador de plano, também em Python, que resolve defaults, aplica policy, consulta a matriz de capacidades do host e produz um `MountPlan` estável. O terceiro é o runtime worker, invocado pelo control plane, que usa Cython para chamar o core nativo. O quarto é o core em C++, que executa o bootstrap e devolve tanto status quanto evidência observada. O quinto é a camada de observabilidade e reconciliação, que persiste eventos, snapshots de `mountinfo` e diagnósticos comparativos entre o plano desejado e a topologia efetiva.

Em termos de metadados, eu manteria pelo menos quatro agregados persistidos: `containers`, `mount_plans`, `mount_observations` e `host_capabilities`. O primeiro guarda identidade, status e relação com o processo ou `pidfd`. O segundo guarda o plano desejado e seu checksum. O terceiro guarda snapshots de `mountinfo`, resultado da verificação e diferenças observadas. O quarto guarda suporte do host a recursos como nova mount API, restrições de perfis rootless, necessidade de fallback e peculiaridades do kernel local. Esse banco não substitui o kernel como fonte de verdade; ele guarda a história e permite reconciliação.

5.2 Desired state, observed state e reconciliação

Esta feature pede explicitamente uma separação entre estado desejado e estado observado. O desired state é o que a API aceita: por exemplo, “rootfs com propagation final `rprivate`”, “bind `/srv/data` em `/data` com `rslave`”, “volumes sempre `rprivate`”. O observed state é o que o runtime encontra em `/proc/<pid>/mountinfo` depois de terminar o bootstrap. O reconciler compara os dois e só então promove o container ao estado `RUNNING`. Se houver divergência, o container entra em `DEGRADED` ou `FAILED`, dependendo da severidade.

Esse desenho é superior a um modelo puramente otimista por dois motivos. Primeiro, o kernel pode degradar ou bloquear operações em perfis menos privilegiados, especialmente com user namespaces. Segundo, o comportamento do host importa: um bind mount que o plano quer como compartilhado só faz sentido se o caminho de origem já suportar propagation compatível. A própria documentação do Docker

ressalta que o host precisa estar configurado adequadamente para bind propagation. Logo, a API não deve mentir; se a capacidade do host não existe, o plano deve ser rejeitado ou degradado de forma explícita, nunca silenciosa [8].

Decisão de produto recomendada

Eu exporia dois perfis operacionais. O perfil *strict-isolation* seria o default e definiria `rprivate` como política externa do rootfs e dos mounts não especiais. O perfil *compat* usaria um bootstrap interno `rslave` e permitiria bindings mais avançados em cenários bem controlados. A API continuaria descrevendo a intenção final; o detalhe de bootstrap seria responsabilidade do executor, versionado por capability profile.

5.3 Admission control e policy de propagation

No seu sistema, propagation não deve ser um campo livre que aceita qualquer string OCI ou Docker-like sem contexto. Ela precisa passar por admission control com regras do tipo: volumes nomeados são sempre `rprivate`; bind mounts só podem pedir `shared` ou `rshared` se a policy do ambiente permitir; perfis `rootless` não prometem `shared`; combinações com `NoPivotRoot` são vetadas quando incompatíveis; e o rootfs padrão do produto só pode sair de `rprivate` mediante escolha explícita e rastreável. Essa camada de policy precisa ficar em Python, onde auditores e desenvolvedores conseguem lê-la facilmente.

Há também uma política importante sobre o host: o runtime não deve remountar caminhos do host como compartilhados por conta própria para “fazer a feature funcionar”. Essa prática altera a semântica do sistema hospedeiro e cria efeitos colaterais difíceis de justificar. Se um caminho do host não atende ao propagation mode exigido, a API deve rejeitar a solicitação e explicar por quê. O runtime só deve ajustar propagation dentro do namespace do container ou em mounts claramente criados e possuídos por ele.

5.4 Sincronização do bootstrap

Eu recomendo um handshake explícito entre pai e filho durante a criação do container. Depois de aplicar mounts e antes de executar o processo `init` do container, o filho nativo sinaliza “mount graph pronto”. Nesse momento, o pai lê `/proc/<pid>/mountinfo`, persiste o snapshot, executa a verificação `desired vs observed` e só então libera o filho para o `execve`. Esse modelo transforma a topologia de mounts em parte do protocolo de criação do container, o que facilita depuração, auditoria e testes diferenciais contra `runc`.

6. Low Level Design

No nível baixo, esta feature deve ser desenhada como uma transação de construção de topologia. O backend nativo recebe um plano imutável e precisa transformar esse plano em uma sequência ordenada de syscalls, com rollback compensatório e um ponto formal de verificação. O design correto aqui não é um conjunto de chamadas soltas para `mount(2)`; é um pipeline com pré-condições, pós-condições e estado intermediário explicitamente modelado.

6.1 Modelos nativos e tipagem explícita

O primeiro cuidado é representar propagation como tipo forte, não como string perdida em um mapa de opções. Isso reduz erros de combinação, evita decisões implícitas e obriga o compilador a participar da segurança do código. Abaixo está um exemplo minimalista de structs nativas que eu usaria para o backend clássico e para o backend moderno. Repare que o `LaunchPlanNative` carrega tanto a intenção final quanto a estratégia de bootstrap, sem misturar uma na outra.

```

enum class PropagationMode {
    Private,
    PrivateRecursive,
    Slave,
    SlaveRecursive,
    Shared,
    SharedRecursive,
    Unbindable,
    UnbindableRecursive
};

struct MountSpecNative {
    std::string source;
    std::string target;
    std::string fstype;           // vazio para bind
    uint64_t mount_flags_set;
    uint64_t mount_flags_clear;
    PropagationMode propagation;
    bool recursive_bind;
    bool read_only;
    bool is_volume;
};

struct LaunchPlanNative {
    std::string container_id;
    std::string rootfs_path;
    PropagationMode final_root_propagation;
    bool strict_private_bootstrap;
    bool no_pivot_root;
    bool rootless_profile;
    std::vector mounts;
};

```

Essa modelagem deixa claro que existe uma diferença entre `final_root_propagation` e `strict_private_bootstrap`. Sem essa distinção, o código rapidamente se torna confuso: qualquer alteração para compatibilidade parece mudar o contrato do produto, e qualquer ajuste de policy parece exigir mudança no algoritmo de bootstrap. Tipagem separada é uma defesa estrutural contra essa confusão.

6.2 Backend clássico baseado em mount(2)

O backend clássico existe porque é amplamente compatível, suficientemente expressivo para a feature e próximo do que runtimes maduros fazem hoje. O algoritmo recomendado segue a mesma disciplina conceitual observada em `runc`, mas com uma decisão de produto mais explícita sobre o default. Em perfil estrito, o filho remounta / como `MS_PRIVATE | MS_REC` logo após entrar no namespace. Em perfil compatível, usa `MS_SLAVE | MS_REC` como estágio inicial. Em ambos os casos, o pai do `rootfs` é tornado privado antes do self-bind. Depois vêm a criação de mount points, a montagem dos pseudo-filesystems, a aplicação dos binds declarados, o `pivot_root` e, por fim, a leitura de `mountinfo` [3,11,12].


```

LaunchResult launch_container(const LaunchPlanNative& plan) {
    Child child = launcher.clone_or_fork();
    child.enter_mount_namespace();

    if (plan.strict_private_bootstrap) {
        child.remount_propagation("/", MS_PRIVATE | MS_REC);
    } else {
        child.remount_propagation("/", MS_SLAVE | MS_REC);
    }

    child.make_rootfs_parent_private(plan.rootfs_path);
    child.bind_mount(plan.rootfs_path, plan.rootfs_path, MS_BIND | MS_REC);

    child.prepare_internal_mountpoints();
    child.apply_declared_mounts_in_order(plan.mounts);

    if (!plan.no_pivot_root) {
        child.pivot_root_into(plan.rootfs_path);
    } else {
        child.chroot_into(plan.rootfs_path);
    }

    child.apply_final_root_propagation_if_needed(plan.final_root_propagation);
    MountSnapshot observed = child.read_mountinfo();
    child.verify_mount_plan(plan, observed);
    child.sync_ready_with_parent(observed);

    return child.exec_container_init();
}

```

Dois pontos do algoritmo merecem destaque. O primeiro é a ordem dos mounts declarados pelo usuário. A especificação OCI exige que mounts sejam aplicados na ordem listada, e vários edge cases dependem disso. O segundo é que flags de propagation dos mounts individuais devem ser aplicadas depois da criação do mount correspondente. Isso também é compatível com a forma como o `runc` organiza sua lógica: primeiro o mount existe, depois sua propagation é ajustada [11,12].

6.3 Tornar privado o mount pai do rootfs

Esse passo é pequeno no código e gigantesco no desenho. A função precisa caminhar para cima no path do rootfs até encontrar o mountpoint correspondente e então remountá-lo como privado. O `runc` trata `EINVAL` como sinal de que o path ainda não é um mountpoint e continua subindo na árvore. A ideia é simples: o self-bind do rootfs só é seguro quando o mount em que ele será criado não pode replicar esse novo bind para fora do namespace [12].

```

void make_rootfs_parent_private(const std::string& rootfs) {
    std::string path = parent_directory(rootfs);

    while (true) {
        if (mount(nullptr, path.c_str(), nullptr, MS_PRIVATE, nullptr) == 0) {
            return; // encontramos um mountpoint e o tornamos private
        }
        if (errno != EINVAL) {
            throw SyscallError("mount(MS_PRIVATE)", path, errno);
        }
        if (path == "/") {
            throw SyscallError("rootfs parent mount not found", path, EINVAL);
        }
        path = parent_directory(path);
    }
}

```

Note que esse algoritmo não é um hack, e sim parte do contrato de isolamento. Ele deve ser encapsulado em uma unidade testável, com logs claros e erro explícito se nenhuma ancoragem de mount for encontrada. Em ambientes com layouts incomuns de storage, esse detalhe é o que separa um runtime que apenas “passa nos testes felizes” de um runtime operacionalmente confiável.

6.4 Backend moderno com a nova mount API

Desde a nova mount API do kernel, existe uma oportunidade de simplificar parte da complexidade histórica de propagation. A documentação de `move_mount(2)` e o texto de Christian Brauner mostram que é possível construir árvores de mounts destacadas, privadas inicialmente, e só depois movê-las para o namespace alvo e ajustar sua propagation. Isso é particularmente atraente para hot-plug de mounts em containers já rodando e para diminuir a dependência de “danças” de shared subtree apenas para inserir um mount em outro namespace [4,13,14].

```

// Visão conceitual do backend moderno
fd = open_tree(AT_FDCWD, source_path, OPEN_TREE_CLONE | AT_RECURSIVE);
mount_setattr(fd, "", AT_EMPTY_PATH | AT_RECURSIVE, {.attr_set =
MOUNT_ATTR_RDONLY});
setns(target_mnt_ns_fd, CLONE_NEWNS);
move_mount(fd, "", AT_FDCWD, target_path, MOVE_MOUNT_F_EMPTY_PATH);

```

Minha recomendação é implementar esse backend desde o dia zero como interface, mesmo que a primeira entrega use apenas o backend clássico. Em termos de design, isso significa definir um contrato como `IMountBackend` com duas implementações: `ClassicMountBackend` e `FdMountBackend`. O runtime escolhe entre elas a partir da matriz de capacidades do host. A API não precisa expor o backend; ela expõe garantias. Essa separação prepara o sistema para evoluir sem quebrar o modelo de domínio.

6.5 Segurança de paths e criação de mount points

Path handling é um dos pontos mais subestimados em runtimes caseiros. O mount target precisa permanecer sob o rootfs do container, e o source de bind mount precisa ser resolvido no host com regras que não permitam truques com symlink, magic links ou caminhos que escapem do diretório esperado. Em kernel novo, eu usaria `openat2` com resoluções restritivas para abrir o destino de forma segura. O runtime então operaria o máximo possível por file descriptor, reduzindo TOCTOU. Se um fallback baseado em caminhos for inevitável, esse fallback precisa ser conscientemente marcado como mais fraco na matriz de capacidades.

Esse é outro motivo para a divisão Python/Cython/C++ ser acertada. O control plane pode validar semanticamente o target, mas a prova final de segurança é nativa e depende de como o backend abre e fixa os diretórios. No design do sistema, mount path safety não é “validação de input”; é parte da fronteira de segurança entre host e container.

6.6 Rollback, erros e protocolo com o pai

O backend precisa manter uma pilha de compensações. Cada mount realizado empilha uma operação inversa, tipicamente um `umount2(MNT_DETACH)` e eventualmente limpeza do diretório auxiliar. Antes do `pivot_root`, o rollback é mais simples; depois dele, precisa considerar a raiz antiga e o novo contexto de paths. Isso pede uma máquina de estados explícita no launcher. Em outras palavras, não tente “lembrar na cabeça” em que ponto o processo está; modele o estágio de bootstrap e faça com que cada etapa conheça suas próprias compensações.

Eu também recomendo um canal de sincronização pai-filho com mensagens estruturadas. O filho informa estágio atual, falha e `errno` em um formato binário estável ou JSON pequeno via pipe. O pai persiste isso como evento de auditoria. Quando há sucesso, o filho envia a janela de inspeção antes do `exec`. Esse protocolo evita logs opacos e torna possível reproduzir o caminho que levou à topologia final.

6.7 Perfis rootless e menos privilegiados

Rootless exige humildade arquitetural. A documentação do kernel afirma que, em mount namespaces menos privilegiados, mounts compartilhados herdados podem ser reduzidos a `slave` e certos atributos podem ficar bloqueados. Logo, o LLD deve carregar a ideia de `CapabilityProfile` até o nível nativo. Em perfil rootless, o backend clássico pode precisar degradar garantias, rejeitar certos modes ou apenas provar a propriedade essencial desta feature: não propagar mounts para fora. O que não pode acontecer é a API prometer `rshared` ou `rprivate` forte sem verificar o que o kernel permitiu materializar [1,4].

Decisão prática

Eu começaria com um perfil rootful estável, com `pivot_root`, `parent-private` e snapshot obrigatório de `mountinfo`. Só depois abriria espaço para rootless, mantendo contratos diferentes e nomes diferentes na API.

Anti-padrão

Não trate rootless como um simples booleano. Ele altera a semântica possível de `propagation`, a chance de `EPERM` e até a interpretação de sucesso parcial. Isso merece um perfil de capacidade próprio.

Por fim, uma nuance adjacente: `propagation` e `read-only` recursivo não são a mesma feature. A documentação do Docker observa que `bind mounts` recursivos em modo `read-only` dependem de kernel recente. Eu manteria esses eixos separados no modelo de domínio para que a matriz de capacidades não contamine a semântica de `propagation` com restrições de `read-only` recursivo [8].

7. Domain-Driven Design

Uma boa parte da dificuldade desse tema desaparece quando o domínio é modelado com linguagem precisa. O erro estrutural mais comum em runtimes caseiros é deixar o modelo de domínio colapsar em torno de “um monte de opções do kernel”. Isso produz APIs opacas e baixa legibilidade do sistema. Em vez disso, a feature deve ser representada como um domínio de isolamento de mounts, em que propagation é uma regra de negócio com implicações de segurança, compatibilidade e auditabilidade.

7.1 Bounded contexts

Eu separaria quatro bounded contexts. O primeiro é *Control Plane*, responsável por receber intenção, aplicar policy e persistir o estado administrativo do container. O segundo é *Storage and Rootfs Resolution*, responsável por localizar rootfs, volumes, snapshots e binds de origem. O terceiro é *Runtime and Namespace*, responsável pela materialização da topologia de mounts no kernel. O quarto é *Observability and Diagnostics*, responsável por snapshots de `mountinfo`, trilha de eventos, comparação desired vs observed e relatórios de conformidade.

Essa separação é útil porque evita que o contexto de runtime “engula” tudo. Por exemplo, decidir se volumes são sempre `rprivate` é uma regra de produto e deve nascer no Control Plane. Já decidir se o host suporta o backend moderno é uma capacidade do contexto de Runtime. O contexto de Observability, por sua vez, não toma decisões de policy, mas é ele que prova que as decisões se materializaram corretamente.

7.2 Entidades, agregados e value objects

O agregado principal é o `Container`, cuja identidade sobrevive a reinícios do processo e cujos estados transitam por fases como `PENDING`, `BOOTSTRAPPING`, `RUNNING`, `DEGRADED` e `FAILED`. O `MountPlan` é outro agregado relevante: depois de aceito pela API, ele deve ser imutável. Isso é importante porque a topologia que o runtime tentou materializar precisa ser auditável e reproduzível. Se o plano pudesse ser alterado em trânsito, comparação e replay perderiam valor.

Como value objects, eu usaria `MountSource`, `MountTarget`, `PropagationMode`, `MountFlags`, `RootfsRef`, `CapabilityProfile` e `NamespaceIdentity`. O traço comum é que eles carregam significado, não identidade própria. Um `MountTarget` válido, por exemplo, não é uma string qualquer; ele já incorpora a regra “deve permanecer sob o rootfs resolvido”. Um `PropagationMode` não é um literal textual; ele sabe se é recursivo, se é permitido para volumes, se requer compatibilidade de host e se é compatível com o perfil atual.

7.3 Serviços de domínio

Os principais domain services seriam `MountPlanCompiler`, `PropagationCompatibilityChecker`, `RootfsIsolationService` e `MountObservationReconciler`. O compilador transforma a intenção do usuário em um plano fechado. O checker confronta o plano com policy e capacidades do host. O serviço de isolamento de rootfs decide qual estratégia de bootstrap usar. O reconciler compara snapshot observado

com o plano e classifica o resultado. Note como nenhum desses serviços precisa conhecer detalhes de erro; eles operam em linguagem ubíqua. Já o backend C++ implementa um anti-corruption layer ao lidar com syscalls.

7.4 Eventos de domínio e invariantes

Eu registraria eventos como `ContainerProvisioningStarted`, `MountNamespaceCreated`, `RootfsParentPrivatized`, `RootfsSelfBound`, `RootPivoted`, `MountPlanVerified` e `ContainerStartFailed`. Esses eventos não servem apenas para observabilidade; eles formam a narrativa de por que um container chegou ao estado atual. Em debugging de isolamento, essa narrativa vale mais que logs genéricos.

As invariantes centrais do domínio, no meu entendimento, são as seguintes, em forma descritiva. Primeiro, um container só pode ser marcado como **RUNNING** depois de a topologia de mounts ser verificada contra o plano persistido. Segundo, o mount target nunca pode escapar do rootfs. Terceiro, volumes nomeados nunca saem de **rprivate**. Quarto, qualquer propagation mais permissiva que **private** exige aprovação explícita da policy e compatibilidade do host. Quinto, a combinação **NoPivotRoot** com rootfs privado é proibida quando a semântica não pode ser garantida. Esses invariantes são regras de negócio sobre isolamento; o kernel apenas é o meio pelo qual elas se tornam realidade.

Resultado prático do DDD aqui

Quando o domínio é modelado dessa forma, a discussão deixa de ser “quais flags eu passo para `mount()`?” e passa a ser “que garantia de isolamento estou oferecendo, em que perfis, sob que evidências?”. Isso é exatamente o tipo de deslocamento conceitual que torna um runtime caseiro sustentável.

8. Padrões de Design (GoF)

Os padrões GoF mais úteis aqui não são ornamento acadêmico; eles ajudam a conter complexidade real. O primeiro padrão que eu adotaria é *Builder* para o `MountPlan`. A API recebe intenção fragmentada do usuário, mas o runtime precisa de um artefato fechado, coerente e validado. O Builder permite acumular decisões, aplicar defaults e produzir um plano imutável sem expor o backend nativo a estados incompletos.

O segundo é *Strategy*, aplicado em duas dimensões. A primeira é a estratégia de propagation do rootfs, distinguindo pelo menos *strict-private* e *compat-rslave*. A segunda é a estratégia de backend de mounts, separando backend clássico e backend baseado em file descriptors. Em ambos os casos, o domínio pede a garantia e o runtime escolhe a técnica concreta. Isso preserva o contrato do produto enquanto permite evolução do executor.

O terceiro é *Command*, especialmente útil para rollback. Cada operação de bootstrap pode ser representada como um comando executável com sua compensação inversa. “Aplicar bind mount”, “montar proc”, “pivotar a raiz”, “aplicar propagation em mount individual” e “criar diretório auxiliar” tornam-se comandos que conhecem a si mesmos e que podem ser desfeitos conforme o estágio da máquina de estados. Em sistemas que manipulam múltiplos mounts por container, essa estrutura reduz muito a chance de limpeza incompleta.

Eu também usaria *Template Method* para o esqueleto do bootstrap. A sequência geral - preparar namespace, ajustar propagation, preparar rootfs, aplicar mounts, trocar raiz, verificar, sincronizar e executar init - tende a permanecer estável. O que varia é a implementação de algumas etapas, dependendo do backend ou do perfil. Um Template Method em C++ deixa essa espinha dorsal explícita, o que ajuda revisão, auditoria e testes.

Facade é útil na fronteira com Python. A camada Cython não deveria expor dezenas de funções de baixo nível; ela deveria chamar um pequeno facade nativo, algo como `RuntimeFacade::launch()`, `RuntimeFacade::inspect_mounts()` e `RuntimeFacade::apply_mount_update()`. Isso reduz o acoplamento e impede que detalhes internos virem parte do contrato binário acidentalmente.

State é apropriado para o ciclo de vida do container e para a máquina de bootstrap. O objeto de contexto pode mudar de comportamento conforme passa por `NamespaceReady`, `RootfsPrepared`, `Pivoted` e `Verified`. Esse padrão vale a pena porque rollback, logging e permissões variam conforme o estágio. Sem ele, o código tende a acumular condicionais frágeis e pouco testáveis.

Por fim, eu aplicaria *Abstract Factory* para montar o conjunto de objetos coerentes com cada perfil de capacidade. Um host com suporte à nova mount API pode receber uma fábrica que instancia backend fd-based, parser avançado, política de hot-plug e certas validações extras. Um host mais conservador recebe a fábrica clássica. O ponto central é impedir combinação arbitrária de peças incompatíveis.

O padrão que eu conscientemente evitaria é *Singleton* para runtime global. Em um sistema com múltiplos launches concorrentes, perfis diferentes e testes paralelos, Singletons encobrem dependências e dificultam isolamento. Também evitaria um “mega objeto” Cython que misture Facade, Builder, Strategy e Estado no mesmo lugar. Nesse projeto, a clareza arquitetural vale mais que a redução superficial de arquivos.

9. Requisitos funcionais e não funcionais

Os requisitos abaixo foram escritos para cobrir tanto o aspecto conceitual da feature quanto o impacto real em engenharia. Eles assumem que o runtime não quer apenas reproduzir chamadas de syscalls, mas oferecer uma garantia operacional de isolamento, auditabilidade e compatibilidade evolutiva.

9.1 Requisitos funcionais

ID	REQUISITO FUNCIONAL
RF-01	Cada container deve iniciar em mount namespace dedicado, criado pelo runtime nativo, com escolha explícita de estratégia de bootstrap de propagation e registro do perfil de capacidade usado na execução.
RF-02	O sistema deve impedir propagação de mounts para fora por padrão. No perfil padrão do produto, isso significa rootfs final <code>rprivate</code> ; em perfis compatíveis, pelo menos a propriedade “não propaga para fora” deve ser garantida e verificada.
RF-03	Bind mounts declarados pelo usuário devem aceitar modos de propagation equivalentes aos do ecossistema Docker/OCI, mas somente quando compatíveis com policy e capacidades do host. A API deve rejeitar explicitamente modos não suportados.
RF-04	Volumes gerenciados pelo runtime devem ser materializados sempre com propagation <code>rprivate</code> , sem possibilidade de override por configuração do usuário, para preservar previsibilidade e alinhamento com a prática do Docker.
RF-05	O backend deve tornar privado o mount pai do rootfs antes do self-bind do rootfs e antes de <code>pivot_root</code> , registrando esse evento como etapa formal do bootstrap.
RF-06	O sistema deve validar combinações inseguras ou semanticamente incompatíveis, incluindo pelo menos cenários equivalentes a <code>NoPivotRoot</code> com rootfs privado, targets fora do rootfs e solicitações de propagation não compatíveis com o perfil rootless.
RF-07	Ao fim do bootstrap, o runtime deve capturar <code>/proc/<pid>/mountinfo</code> , normalizar o snapshot e compará-lo ao plano persistido antes de marcar o container como pronto.
RF-08	Falhas parciais durante o bootstrap devem acionar rollback idempotente, com limpeza de mounts e diretórios auxiliares na medida permitida pelo estágio atingido pela máquina de estados.
RF-09	A implementação deve prever dois backends de execução, um clássico e um backend moderno baseado na nova mount API, com seleção orientada por capability profile.

ID	REQUISITO FUNCIONAL
RF-10	O sistema deve expor uma API de inspeção que mostre desired state, observed state, diferenças encontradas, flags efetivas de propagation e mensagens de diagnóstico orientadas ao operador.
RF-11	Se o produto vier a suportar hot-plug de mounts ou exec em processos dentro do container, o runtime deve preservar ou reconstruir uma referência segura ao mount namespace, como <code>pidfd</code> ou descritor equivalente, de forma rastreável.

9.2 Requisitos não funcionais

ID	REQUISITO NÃO FUNCIONAL
RNF-01	A implementação deve priorizar segurança e menor privilégio, isolando o caminho privilegiado em worker dedicado e evitando operações de namespace dentro do processo Python que serve a API.
RNF-02	O comportamento do runtime deve ser determinístico: o mesmo MountPlan em hosts equivalentes deve produzir topologia observada equivalente, com diferenças devidamente explicadas pela capability matrix.
RNF-03	O sistema deve ser auditável. Cada etapa relevante do bootstrap deve produzir evento estruturado com timestamp, estágio, paths envolvidos, flags, backend utilizado e resultado.
RNF-04	A solução deve ser compatível com heterogeneidade de kernels e ambientes de boot, inclusive hosts com raízes compartilhadas por systemd, perfis rootless limitados e hosts sem suporte à nova mount API.
RNF-05	O custo de criação de container deve permanecer baixo o suficiente para não transformar a verificação de mounts em gargalo; snapshots e parse de <code>mountinfo</code> devem ser lineares e preferencialmente sem alocação excessiva.
RNF-06	O código deve ser testável por camadas. Regras de policy precisam ser cobertas em testes Python; parse e algoritmo de bootstrap, em testes nativos; e o sistema inteiro, em testes diferenciais contra OCI/runc.
RNF-07	Falhas devem ser explicáveis. Mensagens de erro não podem limitar-se a “mount falhou”; devem indicar syscall, <code>errno</code> , path, estágio e regra de domínio violada quando houver.
RNF-08	O design deve permitir evolução incremental sem ruptura da API, preservando a semântica de propagation ao trocar backend clássico por backend moderno.
RNF-09	

ID	REQUISITO NÃO FUNCIONAL
	Observabilidade deve incluir métricas de taxa de falha por estágio, distribuição de modos de propagation usados, divergências desired vs observed e tempo gasto em cada etapa do bootstrap.

Se eu tivesse de resumir esses requisitos em uma única frase, seria esta: a feature não termina quando o mount namespace é criado; ela só termina quando o sistema prova, com evidência observável, que a propagation final está coerente com o contrato assumido. Esse é o fio que conecta todos os requisitos acima.

10. Plano de implementação por fases

Fase 0 - laboratório e matriz de capacidades

A primeira fase deve criar um laboratório de compatibilidade, não escrever o runtime inteiro. O objetivo é medir comportamentos em pelo menos três perfis: host com raiz compartilhada típica de systemd, host mais simples sem essa topologia, e host com suporte à nova mount API. Nessa fase, construa pequenos executáveis nativos que façam apenas `unshare`, `remount` de `propagation`, `self-bind` e leitura de `mountinfo`. O resultado esperado é uma matriz objetiva de capacidade e diferença de comportamento, que depois alimentará o admission control do produto.

Fase 1 - modelo de domínio e persistência do plano

Com o terreno conhecido, implemente em Python o modelo de domínio, o schema de persistência e o `MountPlanBuilder`. Nessa fase, o foco não é executar container de verdade, mas representar corretamente intenções, defaults, policies e invariantes. Volumes como `rprivate`, perfis de `propagation` permitidos e checagens como “target sob o rootfs” devem estar testados antes de qualquer syscall real. O entregável desta fase é a linguagem do produto.

Fase 2 - backend clássico MVP

Na fase seguinte, implemente o backend clássico em C++, chamado a partir de Cython. O MVP deve suportar: criação do mount namespace, perfil padrão `strict-private`, `parent-private` no rootfs, `self-bind`, `pivot_root`, montagem de pseudo-filesystems essenciais, binds do usuário em `rprivate` e snapshot obrigatório de `mountinfo`. Não tente, nessa etapa, suportar todo o espaço OCI. O objetivo é provar o contrato central da feature de modo pequeno e sólido.

Fase 3 - reconciliação e inspeção

Com o backend funcional, acrescente a API de inspeção, o reconciler desired vs observed e os eventos estruturados. Esta fase é o momento em que o sistema deixa de ser apenas executor e passa a ser plataforma operacional. Você quer conseguir perguntar “por que este container ficou degradado?” e receber uma resposta específica: qual mount não bateu, que tags de `propagation` foram observadas e em que estágio ocorreu a divergência.

Fase 4 - modos avançados de propagation e policy admission

Somente depois da fase 3 eu abriria o espaço para `rslave`, `shared` e demais modos avançados em bind mounts específicos. Isso exige admission control mais fino, mensagens de erro melhores e testes mais ricos. Nesta fase também entram perfis `rootless`, mas sempre como perfil separado, nunca como “o mesmo caminho com menos privilégios”.

Fase 5 - backend moderno fd-based

A quinta fase introduz o backend baseado em `open_tree`, `move_mount` e `mount_setattr`. O alvo aqui não é apenas performance, e sim robustez evolutiva. Esse backend abre espaço para inserção de mounts em namespaces já existentes de forma mais limpa, para melhor manipulação por file descriptor e para um futuro de hot-plug mais seguro. O contrato da API deve permanecer o mesmo; apenas a técnica de execução muda.

Fase 6 - hardening e testes diferenciais

A fase final é de endurecimento. Compare o comportamento do seu runtime com `runc` e, quando aplicável, com Docker, usando specs equivalentes e comparando snapshots de `mountinfo`. Faça injeção de falhas em cada estágio do bootstrap, teste rollback, valide comportamentos em hosts com e sem `systemd` e consolide documentação operacional. Só depois disso eu consideraria a feature madura para ser base de outros recursos, como hot-plug de volumes ou execução de processos adicionais em containers já ativos.

11. Estratégia de testes, validação e observabilidade

Em uma feature como esta, testes unitários são necessários, mas insuficientes. O comportamento correto depende da interação entre runtime e kernel, e o kernel carrega topologia herdada do host. Portanto, a estratégia de validação precisa combinar testes por camada com testes diferenciais em ambiente real.

11.1 Testes diferenciais contra runc e OCI

Eu recomendo construir um pequeno gerador de specs equivalentes: de um lado, ele produz seu `MountPlan`; de outro, uma spec OCI comparável para o `runc`. Em seguida, inicialize containers equivalentes e compare snapshots de `mountinfo`. O objetivo não é clonar cada detalhe de implementação, e sim verificar que o comportamento observável das combinações suportadas é compatível com o contrato esperado. Esse tipo de teste é extremamente valioso para detectar regressões discretas de propagation.

11.2 Testes negativos indispensáveis

Os casos negativos devem incluir pelo menos os seguintes cenários. Primeiro, criar um namespace novo sem remount explícito de propagation em um host com raiz compartilhada, para provar que o vazamento é real e para impedir regressão futura em que alguém “simplifique” o bootstrap. Segundo, tentar rootfs privado com caminho equivalente a `NoPivotRoot`, esperando rejeição. Terceiro, tentar target que escape do rootfs via symlink ou caminho malicioso. Quarto, induzir falha intermediária no bootstrap e verificar se nenhum mount residual fica visível no host ou no namespace do filho. Quinto, em perfil rootless, solicitar um modo de propagation incompatível e verificar se a resposta do sistema é clara e específica.

11.3 Observabilidade operacional

Observabilidade aqui não é só log. Ela inclui métricas, eventos estruturados e dados de inspeção. Eu coletaria, pelo menos, tempo por estágio do bootstrap, taxa de falha por syscall, contagem de containers por perfil de propagation, número de divergências desired vs observed e volume de rollbacks bem-sucedidos. Também manteria uma trilha de eventos por container com IDs correlacionáveis entre API, worker e processo filho. Em incidentes de produção, isso economiza horas.

Na camada de inspeção, eu exporia uma visão que relacione cada mount declarado com seu mount observado correspondente, incluindo target, source real, flags de read-only, propagation observada e tags de `mountinfo`. O operador não deveria ser obrigado a interpretar manualmente todo o arquivo `mountinfo` para entender um desvio.

11.4 Critério de pronto

Minha definição de “pronto para uso” para esta feature é severa de propósito. O sistema só está pronto quando consegue provar, em CI e em host real, que um container default não vaza mounts para fora; que perfis compatíveis e avançados respeitam suas próprias regras; que rootless produz mensagens honestas sobre limitações; e que o snapshot de `mountinfo` sempre acompanha a criação bem-sucedida do container. Sem isso, o sistema talvez funcione em demonstração, mas ainda não merece confiança operacional.

12. Conclusão

A feature “mount namespace privado” é um excelente recorte para um runtime estilo Docker porque ela obriga o projeto a enfrentar, desde cedo, a diferença entre isolamento aparente e isolamento semanticamente garantido. O aprendizado central desta pesquisa é que `CLONE_NEWNS` sozinho não resolve o problema. A árvore de mounts é herdada com sua semântica de propagation, e um host com mounts compartilhados continuará compartilhando problemas com o container se o runtime não intervier explicitamente [1,2,3].

No ecossistema original, Docker expõe defaults prudentes, como `rprivate` para bind mounts e volumes, enquanto a cadeia OCI/runc materializa a topologia com uma disciplina interna cuidadosa que inclui `rslave` de bootstrap, privação explícita do mount pai do rootfs, self-bind e `pivot_root`. Para o seu projeto, a melhor síntese dessa experiência é assumir `rprivate` como contrato externo padrão, preservar `rslave` como ferramenta interna e versionar claramente os perfis que alteram esse comportamento [8,9,10,11,12].

Arquiteturalmente, a divisão Python, Cython e C++ faz sentido e, na verdade, reforça a qualidade do resultado se você mantiver a disciplina certa. Python define linguagem de domínio, policy, persistência e reconciliação. Cython é uma ponte estreita. C++ executa syscalls, rollback e prova técnica. Se essa fronteira for respeitada, o resultado não será apenas um runtime que “sobe um container”; será uma base crível para features futuras como volumes mais avançados, hot-plug de mounts, exec em namespaces ativos e compatibilidade evolutiva com a nova mount API do kernel.

Em resumo, a implementação correta desta feature não é um detalhe de kernel escondido no fundo do projeto. Ela é um eixo de arquitetura. Quem a resolve bem, resolve junto boa parte da disciplina de isolamento, observabilidade e design de domínio que diferencia um experimento de um runtime sério.

Observação metodológica: as definições e comportamentos descritos neste relatório se baseiam em documentação canônica do kernel Linux, documentação oficial do Docker, especificação OCI e no código do runc. As escolhas de arquitetura, requisitos e roadmap são propostas de engenharia derivadas dessas fontes para o contexto específico do projeto pedido.

Referências

- [1] mount_namespaces(7). Linux man-pages project. Disponível em: https://man7.org/linux/man-pages/man7/mount_namespaces.7.html. Acesso em 12 mar. 2026.
- [2] Shared Subtrees. Linux kernel documentation. Disponível em: <https://docs.kernel.org/filesystems/sharesubtree.html>. Acesso em 12 mar. 2026.
- [3] mount(2). Linux man-pages project. Disponível em: <https://man7.org/linux/man-pages/man2/mount.2.html>. Acesso em 12 mar. 2026.
- [4] mount_setattr(2). Linux man-pages project. Disponível em: https://man7.org/linux/man-pages/man2/mount_setattr.2.html. Acesso em 12 mar. 2026.
- [5] proc.txt - /proc/*/mountinfo. Linux kernel documentation. Disponível em: <https://www.kernel.org/doc/Documentation/filesystems/proc.txt>. Acesso em 12 mar. 2026.
- [6] Docker Engine overview. Docker documentation. Disponível em: <https://docs.docker.com/engine/>. Acesso em 12 mar. 2026.
- [7] Alternative container runtimes. Docker documentation. Disponível em: <https://docs.docker.com/engine/daemon/alternative-runtimes/>. Acesso em 12 mar. 2026.
- [8] Bind mounts. Docker documentation. Disponível em: <https://docs.docker.com/engine/storage/bind-mounts/>. Acesso em 12 mar. 2026.
- [9] Volumes. Docker documentation. Disponível em: <https://docs.docker.com/engine/storage/volumes/>. Acesso em 12 mar. 2026.
- [10] OCI Runtime Specification - Linux. Open Container Initiative. Disponível em: <https://github.com/opencontainers/runtime-spec/blob/master/config-linux.md>. Acesso em 12 mar. 2026.
- [11] runc - libcontainer/specconv/spec_linux.go. opencontainers/runc. Disponível em: https://github.com/opencontainers/runc/blob/main/libcontainer/specconv/spec_linux.go. Acesso em 12 mar. 2026.
- [12] runc - libcontainer/rootfs_linux.go. opencontainers/runc. Disponível em: https://raw.githubusercontent.com/opencontainers/runc/main/libcontainer/rootfs_linux.go. Acesso em 12 mar. 2026.
- [13] Mounting into mount namespaces. Christian Brauner. Disponível em: <https://people.kernel.org/brauner/mounting-into-mount-namespaces>. Acesso em 12 mar. 2026.
- [14] move_mount(2). Linux man-pages project. Disponível em: https://man7.org/linux/man-pages/man2/move_mount.2.html. Acesso em 12 mar. 2026.
- [15] systemd.exec(5). Linux man-pages project. Disponível em: <https://man7.org/linux/man-pages/man5/systemd.exec.5.html>. Acesso em 12 mar. 2026.